SMU Lyle School of Engineering



Reinforcement Learning

n-step Bootstrapping

Sutton/Barto* Chapter 7

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction, 2nd edition, MIT Press, Cambridge, MA, 2018



Online Material

CC BY NC SA

Topics of this Course

- Introduction to reinforcement learning
- Markov decision processes
- **Part I: Tabular Methods**
  - Dynamic programming
  - Monte Carlo methods
  - Temporal-difference learning
  - **Multi-step bootstrapping**
  - Planning and learning with tabular methods
- Part II: Approximate Solution Methods
  - Prediction and Control using Approximation
  - Eligibility Traces
  - Policy Gradient Methods
- Part III: Modern RL Methods
  - Deep Reinforcement Learning
  - Current Applications

Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\boldsymbol{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\boldsymbol{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bolt)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X, i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\text{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or v_*

MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r \mid s, a)$	probability of transition to state s' and receiving reward r, from state s taking action a.
$p(s' \mid s, a)$	probability of transition to state s' fom state s taking action a.
$r(s, a)$	expected immediate reward from state s after action a.
$r(s, a, s')$	expected immediate reward from state s to s' with action a.
$\pi(a \mid s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

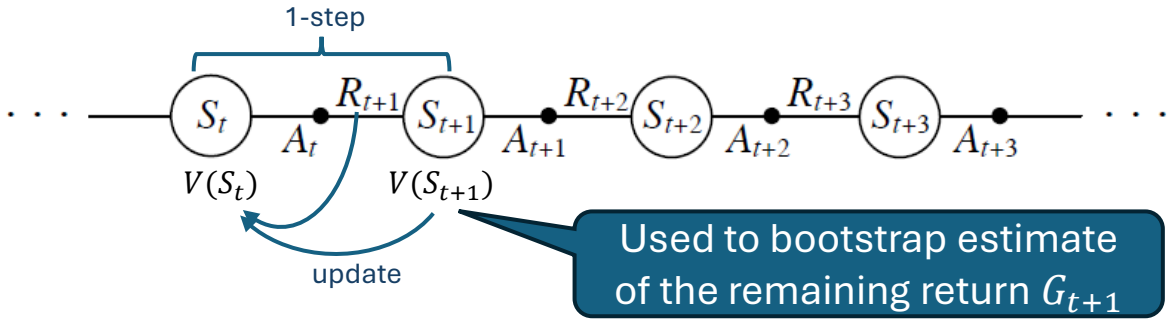
Temporal Difference Learning

U_t	target for estimate at time t
δ_t	TD error
$G_{t:t+n}$	n-step return using rewards from $t+1$ to $t+n$ and bootstrap the rest

Policy Evaluation: 1-step TD vs. n-step TD vs. MC

Question: How far ahead in the sampled episode do we look to estimate the return?

- **MC:** We use the whole episode.
- **1-step TD:** We only observe the **next step** and its reward. The remaining return is **bootstrapped**. This **propagates state value errors**!



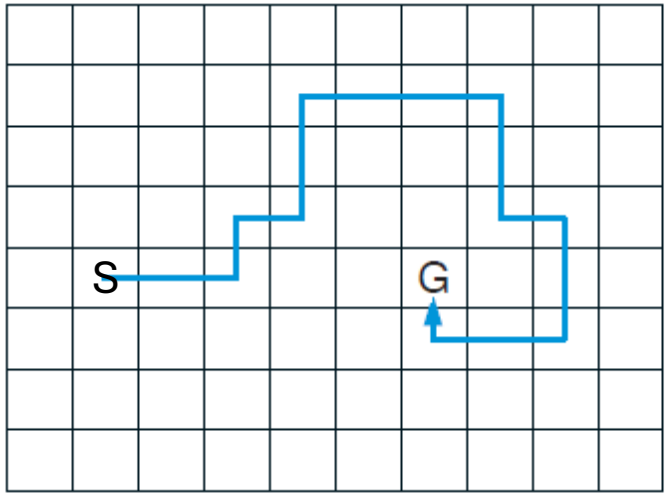
- **n-step TD**
  - Looks n time steps ahead and bootstraps the rest. Using more observed rewards should help to **reduce the bootstrapping error.**
  - TD unifies and generalizes MC and one-step TD.



Why is a Longer Look-ahead Useful?

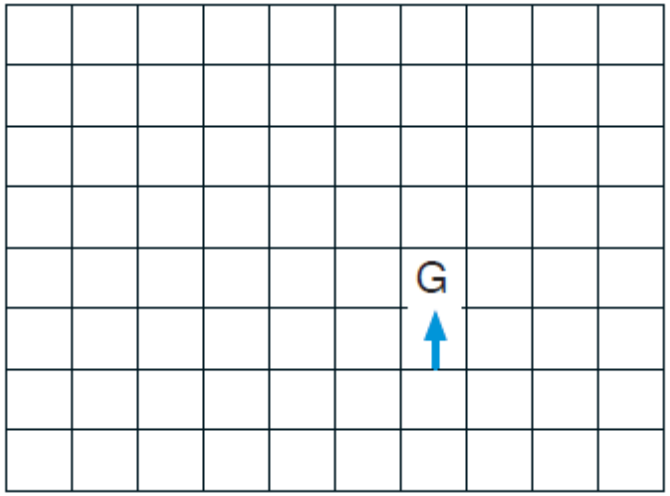
- **Example**: Maze with only one final reward for reaching the goal.
  We learn from the **first episode**.

Episode (MC)



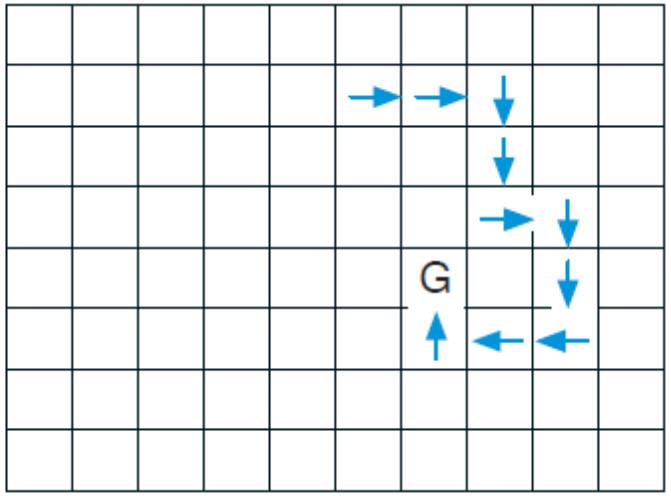
Updates in reverse order all state values on the path using the goal reward.

1-step look ahead



Can only update one state value with the goal reward.

10-step look ahead

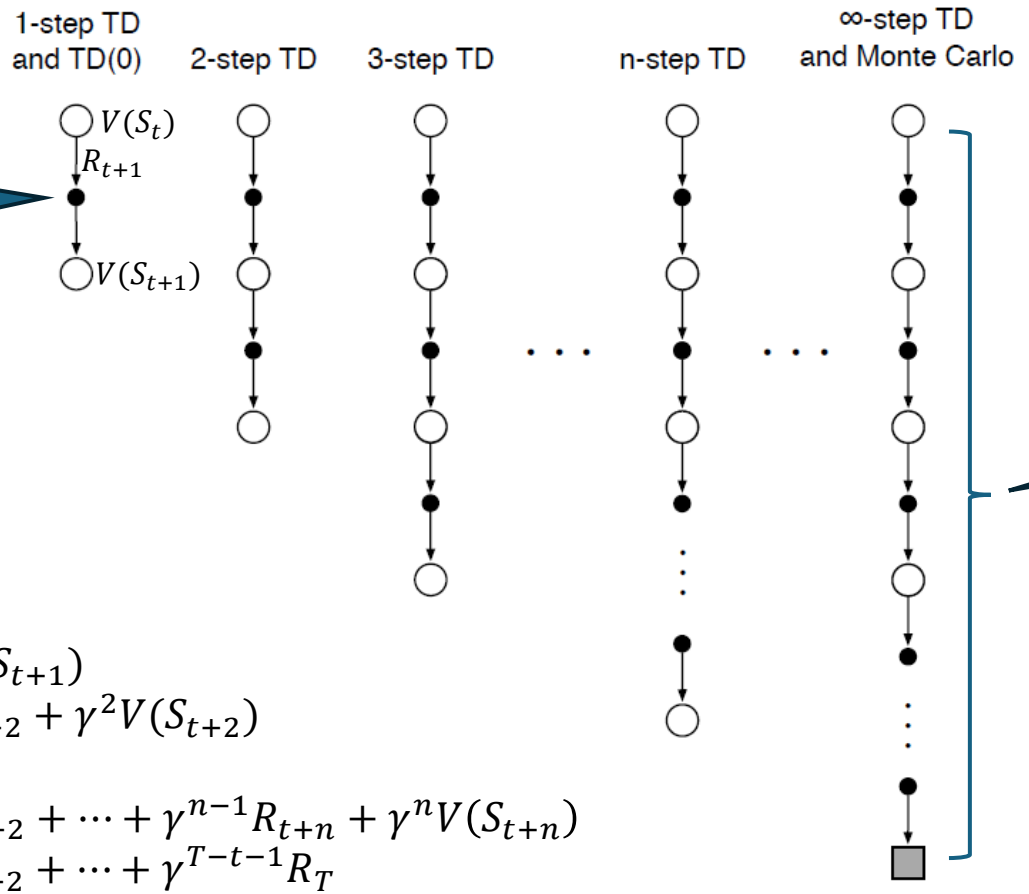


State value can be updated with the goal reward even if it is up to 10 steps away.
Learns faster than 1-step TD.

n-step TD Prediction

Estimate the value function for a given policy.

n-step TD Prediction



Update with reward R_{t+1} + bootstrap the remaining return G_{t+1} by the value of the next state $V(S_{t+1})$.

Update with the whole sequence of observed rewards → no bootstrapping.

Return Estimate

1-step TD: $G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1})$

2-step TD: $G_{t:t+2} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$

...

n-step TD: $G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$

MC: $G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$

Update: $V(S_t) \leftarrow V(S_t) + \alpha[G_{t:t+n} - V(S_t)]$

The update needs $G_{t:t+n}$ and is n steps delayed!

Notation

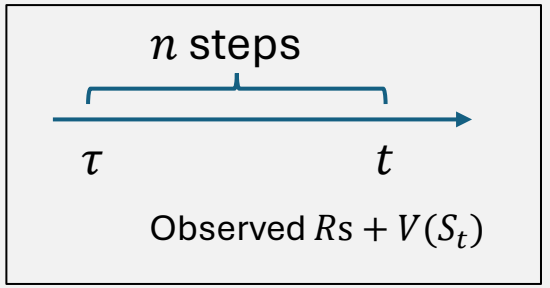
$G_{t:t+n}$ ... the n-step return is calculated from the rewards up to $t+n$ and the rest is bootstrapped using the value function.

Alg. n-step TD Prediction

n-step TD for estimating $V \approx v_\pi$

Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can take their index mod $n + 1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take an action according to $\pi(\cdot|S_t)$
      Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
      If S_{t+1} is terminal, then $T \leftarrow t + 1$
    $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
    If $\tau \geq 0$:
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$
      $V(S_\tau) \leftarrow V(S_\tau) + \alpha\,[G - V(S_\tau)]$
  Until $\tau = T - 1$



Detect the end of the episode

Start only when we can update $t \geq n$

The update is delayed n steps

Use bootstrapping if the episode is not over.
Note: $V(S_{\tau+n})$ is called $V(S_{t+n})$ in the equations!

Error Reduction Property and Convergence

Error Reduction Property of n-step Returns

The expected value of n-step returns is always better than the current approximation (for the worst state).

$$\max_s |\mathbb{E}_\pi[G_{t:t+n}|S_t = s] - v_\pi(s)| \leq \gamma^n \max_s |V(s) - v_\pi(s)|$$

Explanation $G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$

- The averages of observed rewards have in expectation no error (are unbiased).
- The impact of the error-prone bootstrap estimate is smaller because it is n time steps away and thus discounted by γ^n (related to γ-contracting property of Bellman updates).
- Expectation just means we need many estimates (= use many episodes)

Implication: All n-step methods (including 1-step and MC) converge!

Bias-Variance Tradeoff in RL

- **Bias:** The error that occurs when an estimator systematically deviates from the true expected value. High bias means the target relies on a guess or bootstrapped estimate rather than actual ground truth.
- **Variance:** How much the value or return estimate fluctuates due to the randomness (stochasticity) of trajectories, actions, and environments. High variance means a small change in sample episodes leads to wildly different updates.

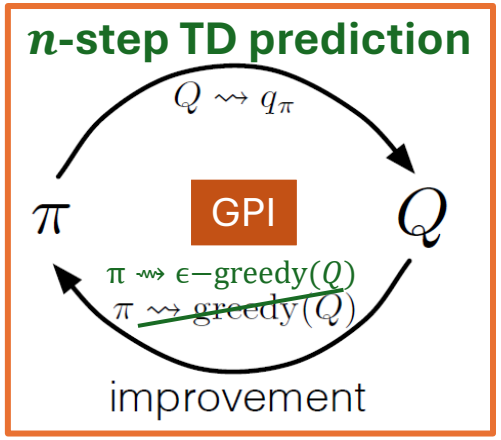
Method	Bias	Variance
MC	**Unbiased –** only uses actual observed rewards.	**High -** accumulating rewards across long, stochastic trajectories creates massive fluctuations.
TD	**High -** bootstrapping uses outdated value estimates for the next states	**Lower -** looking only one step ahead shrinks the variance

n-step Control: Sarsa

On-Policy learning using n-step return estimates.

n-step Sarsa

Note: (0) is not the number of steps but means that $\lambda = 0$. We will learn about eligibility traces later.



Original Sarsa is called one-step Sarsa or Sarsa(0)

n-step Sarsa

- **On policy**: The policy needs to keep exploring (e.g., an ϵ-greedy policy)
- Should probably be called Sarrrrr…rsa with n rs.

This is the extra action Sarsa need.

- n-step return estimate used as the target (redefined using Q):

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^{n} Q(S_{t+n}, A_{t+n})$$

- Update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G_{t:t+n} - Q(S_t, A_t)]$$

Speedup of n-step Sarsa

Q-values updated during one episode

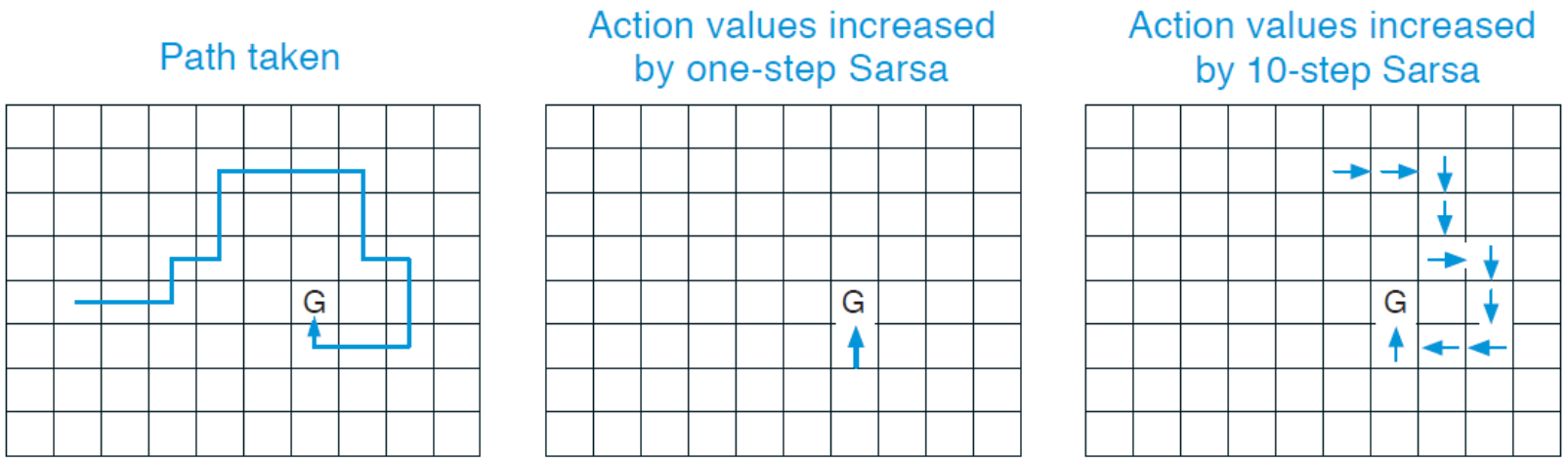


Figure 7.4: Gridworld example of the speedup of policy learning due to the use of n-step methods. The first panel shows the path taken by an agent in a single episode, ending at a location of high reward, marked by the G. In this example the values were all initially 0, and all rewards were zero except for a positive reward at G. The arrows in the other two panels show which action values were strengthened as a result of this path by one-step and n-step Sarsa methods. The one-step method strengthens only the last action of the sequence of actions that led to the high reward, whereas the n-step method strengthens the last n actions of the sequence, so that much more is learned from the one episode.

Alg. From DT prediction to n-step Sarsa Control

n-step TD for estimating $V \approx v$

Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can take their index mod

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take an action according to $\pi(\cdot|S_t)$
      Observe and store the next reward as R_{t+1} and the next state a
      If S_{t+1} is terminal, then $T \leftarrow t + 1$
    $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being upda
    If $\tau \geq 0$:
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$
      $V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$
  Until $\tau = T - 1$

π is given

Update V

n-step Sarsa for estimating $Q \approx q_*$ or q_π

Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be ε-greedy with respect to Q, or to a fixed given policy
Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n + 1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim \pi(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take action A_t
      Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
      If S_{t+1} is terminal, then:
        $T \leftarrow t + 1$
      else:
        Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$
    $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
    If $\tau \geq 0$:
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
      $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$
      If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ε-greedy wrt Q
  Until $\tau = T - 1$

On-policy: Select next action from the current π

Update Q and π

n-step Off-Policy Control

Add importance sampling ratios to prediction and control.

n-step Off-Policy Control

- **Goal**: Learn a deterministic target policy π using an exploring (soft) behavioral policy b.
- **Method**: Use importance sampling ratio as a weight for updates (like in off-policy MC)

$$V_{t+1}(S_t) = V(S_t) + \alpha\rho_{t:t+n-1}[G_{t:t+n} - V(S_t)]$$

where the weight is

$$\rho_{t:h} = \prod_{k=t}^{\min(h,T-t)} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$$

How likely is π to choose an action compared to b?

- Can be used in n-step TD for prediction or in n-step Sarsa for control.

- **Sample efficiency issue**: Can only learn (i.e., $\rho > 0$) from data where the sampled actions for the n-steps are compatible with π.

  Using a behavior policy $b = \epsilon\text{—greedy}(\pi)$ with a small ϵ helps to produce useful observations.

Alg. Comparing On-Policy with Off-Policy Sarsa

n-step Sarsa for estimating $Q \approx q_*$ or q_π

Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be ε-greedy with respect to Q, or to a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, small $\varepsilon > 0$, a positive integ
All store and access operations (for S_t, A_t, and R_t) can take their inde

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim \pi(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take action A_t
      Observe and store the next reward as R_{t+1} and the next state
      If S_{t+1} is terminal, then:
        $T \leftarrow t+1$
      else:
        Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$
    $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
    If $\tau \geq 0$:
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$
      $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\,[G - Q(S_\tau, A_\tau)]$
      If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ε-greedy wrt
  Until $\tau = T - 1$

On-Policy

Off-policy n-step Sarsa for estimating $Q \approx q_*$ or q_π

Input: an arbitrary behavior policy b such that $b(a|s) > 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be greedy with respect to Q, or as a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim b(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take action A_t
      Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
      If S_{t+1} is terminal, then:
        $T \leftarrow t+1$
      else:
        Select and store an action $A_{t+1} \sim b(\cdot|S_{t+1})$
    $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
    If $\tau \geq 0$:
      $\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n,T-1)} \frac{\pi(A_i|S_i)}{b(A_i|S_i)}$ ($\rho_{\tau+1:\tau+n}$)
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
      $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \rho\,[G - Q(S_\tau, A_\tau)]$
      If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q
  Until $\tau = T - 1$

Add behavior policy

Greedy target policy

Popular choice: $b = \epsilon$−greedy(π)

Use importance sampling ratio

n-step Q-Learning

- Off-policy Q-learning with n-step bootstrapping needs correction.

$$G_{t:t+n} = \underbrace{R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n}}_{\text{from the behavior policy}} + \underbrace{\gamma^n \max_a Q(S_{t+n}, a)}_{\text{from the target policy}}$$

- The first action/reward are fine since we will use it to update $Q(s, a)$. R_{t+2} to R_{t+n} and S_{t+n} need to be corrected!

- Options:
  - Only use updates when the all actions chosen by b match the target policy π. This is very sample inefficient.
  - Use importance sampling to correct R_{t+2} to R_{t+n} and S_{t+n}. This introduces large variance.





What you Need to Know

n-step methods let us choose the amount of bootstrapping from 1-step to no bootstrapping (MC).

- Advantage over one-step methods:
  - **Learning**: Looking more steps ahead speeds up learning, especially for sparse rewards.
  - **Error reduction**: Significantly reduces the bootstrapping error (**bias**) at the cost of increased **variance**.
  - An intermediate amount of bootstrapping typically **performs better** in terms of the variance/bias tradeoff than the one-step and MC extremes. n is application-dependent and can be tuned.
  - **DRL**: n-step methods are popular for deep RL!

- Drawback:
  - **Delayed Updates**: delayed n-steps since future events (rewards and states) are needed.
  - **Computation**: More computation per time step and higher memory needs than one-step methods.

    Eligibility traces address the delay and computation drawbacks, but they don't work well with DRL. We will cover these methods later.